DEPLOYMENT PLAN

RAILMIND — LRT AI Operations Decision Support System

UMHackathon 2026 | Final Round

Document Title	Deployment Plan — RAILMIND LRT AI Operations Decision Support System
Version	3.0 (Final — Streamlit Cloud)
Date	3 May 2026
Prepared By	Wi-Five
Repository	https://github.com/kahhhhjun/lrt-ai-operations
Target Environment	Streamlit Community Cloud — https://streamlit.io/cloud
Deployed Link	https://lrt-ai-operations-asaqmnlv7yfpy76tddwbdi.streamlit.app/
Entry Point	auth.py
Status	Final

Prepared for the UMHackathon 2026 judging panel. Addresses judging criteria: Technical Stack & Scalability, CI/CD Pipeline & Automation, Integration Security & Credential Management.

1. Technical Stack & Scalability

1.1 Current Technical Stack

RAILMIND is a Python-based AI decision support system deployed on Streamlit Community Cloud. The table below documents every component of the current production stack, its role, and its scalability classification.

Layer	Component / Technology	Version	Role in System	Scalability Classification
Presentation	Streamlit	>=1.30	Web UI framework. Renders the schedule table, option cards, GLM reasoning panel, cost metrics, and all interactive widgets.	Vertical only — single-process server. Horizontal scaling requires container orchestration (see Section 1.3).
Presentation	Altair	latest	Declarative charting library. Renders load factor chart, frequency comparison, and weekly cost overview.	Stateless — scales with Streamlit.
AI Reasoning	Z.AI GLM-5.1	API	Primary LLM. Streaming reasoning, scheduling recommendation, and shift briefing generation. ~8–14s response time.	Externally managed. Scales independently of the application. Z.AI rate limits apply.
AI Extraction	Z.AI GLM-5-Turbo	API	Lightweight LLM for fast structured extraction of weather, event, and emergency type from free text and images. ~1–2s response time.	Externally managed. Scales independently.
Crowd Detection	HuggingFace DETR	API	facebook/detr-resnet-50 object detection. Counts persons in CCTV images to estimate per-hour passenger density.	Externally managed. Free tier subject to cold-start latency (~20–30s first call).
Business Logic	Python 3.12	3.12+	Scheduling engine (core/calculator.py), orchestration (core/recommender.py), GLM client (core/glm_client.py), DB layer (core/database.py).	CPU-bound computation is lightweight (<50ms per compute_options call). Scales vertically.
Data	pandas	>=2.0	DataFrame construction for schedule display, weekly summary aggregation, and cost calculations.	In-memory. Sufficient for single-user session data volumes.
Persistence	SQLite 3	built-in	Schedule storage (lrt_schedules.db) and user credentials (users.json) via core/database.py.	Single-writer. Not suitable for concurrent multi-user writes. Production replacement: PostgreSQL (see Section 1.3).
HTTP Client	requests	>=2.31	All outbound HTTP calls: Z.AI GLM API, HuggingFace DETR API.	Synchronous. At scale, replace with httpx + asyncio for concurrent API calls.

Layer	Component / Technology	Version	Role in System	Scalability Classification
Image	Pillow	>=10.0	Image pre-processing for CCTV upload and event poster OCR features.	Stateless. Scales with Python process.
Auth	SHA-256 + JSON	built-in	Staff registration and login. Passwords hashed with SHA-256. Session state managed via st.session_state.	Single-file JSON store. Not suitable for concurrent writes at scale. Production replacement: bcrypt + PostgreSQL users table.
Config	Streamlit Secrets	native	API credentials stored in Streamlit Cloud's encrypted secrets store. Read via st.secrets at runtime.	Platform-managed. No scalability concern.

1.2 Current Scalability Characteristics

The following analysis documents the system's behaviour under increasing load. This is honest documentation of a hackathon prototype — the constraints are known and the production upgrade path is defined in Section 1.3.

Concurrent Users

Streamlit Community Cloud runs the application as a single Python process. Each browser session creates an independent Streamlit session with its own st.session_state. The scheduling computation (compute_options, compute_daily_schedule) is CPU-bound but completes in under 50 milliseconds, so contention between concurrent sessions for CPU is negligible.

The binding constraint is the GLM API: each analysis request fires one GLM-5-Turbo call (~1–2s) and one GLM-5.1 streaming call (~8–14s). Under the Z.AI free tier, concurrent GLM calls from multiple sessions will queue at the API level. For the hackathon demo with one active evaluator at a time, this is not an issue. For production with 20+ concurrent duty managers across three lines, a dedicated Z.AI API tier with higher rate limits would be required.

Database Concurrency

SQLite uses file-level locking. Concurrent writes from multiple sessions (e.g. two duty managers saving a schedule at the same instant) will result in one write being queued. For a single-station duty manager tool this is acceptable. For a system serving all three LRT lines simultaneously with multiple operators per shift, PostgreSQL with row-level locking is required.

Memory

Streamlit Community Cloud provides 1 GB RAM. The entire RAILMIND application — including a fully loaded schedule DataFrame, Altair chart data, and GLM response buffers — consumes approximately 80–120 MB per session. Under normal hackathon demo conditions (1–3 concurrent sessions), memory usage is well within the platform limit.

Streamlit Cloud Platform Limits

Resource	Free Tier Limit	RAILMIND Usage	Headroom
CPU	1 vCPU shared	< 5% per scheduling computation	High — compute is lightweight
RAM	1 GB	~80–120 MB per active session	High — 8+ concurrent sessions before concern
Storage	Ephemeral only	SQLite DB + users.json (< 5 MB total)	Sufficient for demo; ephemeral resets on restart
Concurrent apps	1 active app	1 app deployed	No constraint
Bandwidth	Unlimited (shared)	GLM streaming ~5–15 KB/response	No constraint at demo scale
App sleep	After ~7 days idle	N/A — kept active during demo period	Wake time ~20–30s if cold

1.3 Production Scalability Upgrade Path

The following table defines the specific upgrade each component requires to support production-scale deployment across all three LRT lines (Kelana Jaya, Ampang, Sri Petaling) with concurrent multi-shift access.

Component	Current (Prototype)	Production Upgrade	Rationale
Hosting	Streamlit Community Cloud (free, single-process)	Docker container on AWS ECS / Azure App Service, or Streamlit in Snowflake	Enables horizontal scaling, private network, and SLA-backed uptime.
Database	SQLite (file, single-writer, ephemeral on cloud)	PostgreSQL (e.g. AWS RDS, Supabase) with connection pooling via pgBouncer	Supports concurrent writes, persistent storage, queryable history, and schema migrations.
Auth	SHA-256 + JSON file	bcrypt/Argon2 hashing + PostgreSQL users table + session JWT tokens	Eliminates rainbow table vulnerability. Supports password expiry and role-based access.
GLM API tier	Z.AI free / standard tier	Z.AI enterprise tier with dedicated rate limits	Required for >5 concurrent users firing GLM calls simultaneously.
HTTP calls	Synchronous requests library	Async calls via httpx + asyncio, or background task queue (Celery + Redis)	Prevents UI blocking on slow GLM responses; enables parallel extraction + reasoning.
Secrets	Streamlit Secrets (dashboard)	AWS Secrets Manager or HashiCorp Vault with automated rotation	Centralised, auditable, rotatable credential management with access logging.
Ridership data	Hardcoded baselines in calculator.py	Live feed from Prasarana / RapidKL gate-count telemetry API	Eliminates synthetic data inaccuracy; enables real-time demand response.
Monitoring	None	Datadog / Sentry for error tracking, API latency, and fallback activation alerts	Essential for a safety-adjacent public transport operations tool.

2. CI/CD Pipeline & Automation

2.1 Current CI/CD Pipeline (Live)

RAILMIND uses a two-stage CI/CD pipeline: a GitHub Actions workflow for automated testing on every pull request, and Streamlit Cloud's built-in continuous deployment for automatic redeployment on every push to main.

The pipeline enforces a branch protection model: no code reaches the main branch without passing the automated test suite. This prevents broken code from triggering a redeployment and ensures the live application always reflects a tested state.

2.2 Pipeline Architecture

The following describes the full journey of a code change from development to production:

Stage	Trigger	Actions Performed	Gate
1	**Developer pushes to feature branch**	No automated action. Developer runs test_glm.py locally to verify API connectivity before opening a PR.	Manual — developer responsibility
2	**Pull request opened against main**	GitHub Actions workflow (ci.yml) triggers automatically: • Install dependencies from requirements.txt • Run unit tests: python -m pytest tests/ -v • Check for .env files accidentally staged • Validate requirements.txt for pinned versions	*BLOCKING — PR cannot be merged if any check fails*
3	**PR reviewed and merged to main**	GitHub Actions runs the full test suite once more on the merged commit to catch any integration issues introduced by the merge.	*BLOCKING — merge commit must pass*
4	**Streamlit Cloud detects push to main**	Streamlit Cloud automatically: • Clones the updated repository • Installs packages from requirements.txt • Starts the application with the new code • Serves the updated app at the public URL	Automatic — no manual deploy step required
5	**Post-deployment**	Streamlit Cloud logs are available in the dashboard. Any startup error (import failure, missing dependency) is visible immediately in the Manage app → Logs view.	Manual monitoring — check logs after each deploy

2.3 GitHub Actions Workflow Definition

The following is the GitHub Actions workflow file (.github/workflows/ci.yml) that implements Stage 2 and Stage 3 of the pipeline. This file is committed to the repository and runs automatically on every pull request targeting main.

```
name: RAILMIND CI

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]
```

```
jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python 3.12
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r lrt_ai/requirements.txt
          pip install pytest

      - name: Check no .env file is committed
        run: |
          if find . -name '.env' -not -path './.git/*' | grep -q '.'; then
            echo 'ERROR: .env file detected in repository. Remove it
immediately.'
            exit 1
          fi

      - name: Run unit tests
        run: |
          cd lrt_ai
          python -m pytest tests/ -v --tb=short
        env:
          GLM_API_KEY: ${{ secrets.GLM_API_KEY }}
          HF_API_KEY:  ${{ secrets.HF_API_KEY }}
```

Note: GLM_API_KEY and HF_API_KEY are stored as GitHub Actions repository secrets (Settings → Secrets and variables → Actions). They are injected into the CI environment at runtime and are never exposed in logs or workflow output.

2.4 Branch Protection Rules

The main branch is protected with the following rules configured in the GitHub repository settings (Settings → Branches → Branch protection rules):

Rule	Setting	Purpose
Require status checks to pass before merging	Enabled — RAILMIND CI (ci.yml) must pass	Prevents broken code from reaching main and triggering a production redeployment.
Require branches to be up to date before merging	Enabled	Ensures the CI runs against the merged state, not just the feature branch alone, catching integration conflicts.
Require pull request before merging	Enabled — at least 1 review	Enforces code review. No direct pushes to main allowed.
Restrict who can push to matching branches	Team members only	Prevents accidental direct commits to production branch.
Do not allow bypassing the above settings	Enabled	Even repository administrators cannot bypass branch protection.

2.5 Automated Test Coverage in the Pipeline

The GitHub Actions CI pipeline runs the following automated test suite on every pull request. These are the same tests documented in the QATD but executed automatically without human intervention.

Test File	What It Tests	Runs in CI?	Requires Live API?
tests/test_calculator.py	Unit tests for compute_options(), compute_daily_schedule(), _baseline_pax(), _apply_option_to_window(). All deterministic — no external dependencies.	Yes — every PR	No
tests/test_recommender.py	Unit tests for option selection logic, cost aggregation, and the math-based fallback recommendation path.	Yes — every PR	No
tests/test_database.py	Integration tests for SQLite CRUD: init_db(), save_schedule(), load_schedule(), delete_schedule(). Uses an in-memory SQLite database (:memory:) — no file I/O.	Yes — every PR	No
tests/test_auth.py	Unit tests for SHA-256 password hashing, user registration duplicate detection, and login credential validation against a temporary users.json fixture.	Yes — every PR	No
test_glm.py	Live connectivity test for the Z.AI GLM API endpoint. Confirms HTTP 200 and valid response. Requires GLM_API_KEY to be set.	Yes — with secret	Yes (GLM)

Note: test_glm.py runs in CI only when the GLM_API_KEY GitHub Actions secret is configured. If the secret is absent (e.g. on forks or public PRs from external contributors), this test is skipped gracefully — it does not block the PR merge.

2.6 Deployment Automation — Streamlit Cloud

Streamlit Community Cloud implements continuous deployment natively. Once the repository is connected to a Streamlit Cloud app, every push to the monitored branch (main) triggers an automatic redeployment with no manual steps required. The deployment process is as follows:

1. Streamlit Cloud detects the new commit on main via GitHub webhook.
2. The platform clones the updated repository into an isolated build environment.
3. pip install -r requirements.txt is executed automatically to install or update dependencies.
4. The application is started with streamlit run lrt_ai/auth.py.
5. If startup succeeds, traffic is routed to the new instance. If startup fails (import error, missing dependency), the previous version continues serving traffic and an error is shown in the dashboard logs.

Secrets configured in the Streamlit Cloud dashboard persist across redeployments automatically. They do not need to be re-entered unless the key values change. This means a standard code update — push to main, CI passes, Streamlit redeploys — requires zero manual intervention from the moment of git push.

3. Integration Security & Credential Management

3.1 Threat Model — What We Are Protecting

RAILMIND integrates with two external AI APIs (Z.AI GLM and HuggingFace) and maintains a staff credential store. The following assets require protection:

Asset	Classification	Consequence of Exposure
GLM_API_KEY (Z.AI)	Secret — High	Unauthorized parties could consume the API quota at cost to the account holder, or use the key to make API calls impersonating RAILMIND.
HF_API_KEY (HuggingFace)	Secret — Medium	Unauthorized parties could consume the free inference quota, disabling the CCTV crowd detection feature.
Staff passwords	Secret — High	Compromised credentials allow unauthorized access to the scheduling system. In a real deployment, this could enable malicious schedule tampering.
data/users.json	Sensitive — Medium	Contains staff IDs and hashed passwords. Exposure enables offline brute-force attacks against the SHA-256 hashes.
data/lrt_schedules.db	Sensitive — Low	Contains saved schedule records. Exposure reveals operational scheduling patterns but does not directly enable system access.

3.2 API Credential Management — Current Implementation

3.2.1 What Is NOT Done (and Why That Matters)

The following practices, which represent common credential mishandling mistakes, are explicitly avoided in RAILMIND:

Prohibited Practice	RAILMIND Status	Control in Place
API keys hardcoded in source code	Never done	.gitignore excludes .env. GitHub Actions secret scan (Step 2.3) checks for .env files on every commit. Code review required before merge.
.env file committed to repository	Never done	.env and .env.example are listed in .gitignore. The CI pipeline includes an explicit check that fails the build if any .env file is detected in the repository tree.
API keys printed to logs or displayed in UI	Never done	glm_client.py uses the key only in the Authorization header of outbound HTTP requests. The key is never passed to Streamlit UI components or written to any log output.
Secrets shared in chat, email, or plain text	Never done	Keys are entered exclusively through the Streamlit Cloud Secrets dashboard (HTTPS, access-controlled) or GitHub Actions Secrets (encrypted at rest, never shown in logs).
Single shared key used for all environments	Avoided	Local development uses keys from .env (developer's own account). Streamlit Cloud deployment uses keys

Prohibited Practice	RAILMIND Status	Control in Place
		from the Streamlit Secrets store. The two environments are independently keyed.

3.2.2 Streamlit Cloud Secrets — How It Works

On Streamlit Community Cloud, all API credentials are stored using the platform's built-in Secrets management system. The following describes the security properties of this system:

- Secrets are stored encrypted at rest by Streamlit Cloud (Snowflake infrastructure). They are not stored in the repository, in build artifacts, or in deployment logs.
- Secrets are injected into the application runtime as environment variables accessible via st.secrets. They are never written to disk on the Streamlit Cloud host.
- Access to the Secrets dashboard is controlled by the Streamlit Cloud account login (GitHub OAuth). Only the account owner can view or modify secrets.
- Secrets are not visible to GitHub collaborators, even those with write access to the repository.
- Secrets persist across redeployments. Updating a secret value triggers an automatic application restart to pick up the new value.

3.2.3 The _secret() Helper — Dual-Environment Credential Loading

The credential loading logic in core/glm_client.py uses a _secret() helper function that reads from st.secrets first (Streamlit Cloud) and falls back to os.getenv() (local development with .env file). This design ensures identical application behaviour in both environments without any code change between deployments.

```
def _secret(key, default=None):
    """
    Read a secret from Streamlit Cloud (st.secrets) in production,
    or from os.getenv / .env in local development.
    Never raises — returns default if key is absent in both sources.
    """
    try:
        return st.secrets[key]          # Streamlit Cloud: encrypted secrets
store
    except (KeyError, AttributeError, FileNotFoundError):
        return os.getenv(key, default)  # Local: .env file via python-dotenv

# All credentials loaded through the same controlled interface
GLM_API_KEY    = _secret('GLM_API_KEY')
HF_API_KEY     = _secret('HF_API_KEY')
GLM_ENDPOINT   = _secret('GLM_ENDPOINT',
'https://api.z.ai/api/anthropic/v1/messages')
GLM_MODEL      = _secret('GLM_MODEL',    'glm-5.1')
GLM_MODEL_FAST = _secret('GLM_MODEL_FAST','glm-5-turbo')
```

Note: The try/except catches three distinct failure modes: KeyError (key not in secrets), AttributeError (st.secrets object not available — e.g. running outside Streamlit), and FileNotFoundError (Streamlit cannot find a secrets.toml file locally). In all three cases, the fallback to os.getenv is silent and correct.

3.3 API Credential Flow Diagram

The following table traces the lifecycle of the GLM_API_KEY from storage to use, documenting every point where the credential is handled:

Step	Location	Action	Security Control
1	Developer machine	Developer adds GLM_API_KEY to local .env file	.env is in .gitignore — never committed. File exists only on developer's local machine.
2	GitHub repository	Developer does NOT commit .env. CI pipeline checks for .env on every PR.	Automated .env detection in ci.yml. PR blocked if .env file found anywhere in repo tree.
3	GitHub Actions	CI test run injects GLM_API_KEY from GitHub Actions secret into test environment	Secret encrypted at rest in GitHub. Never shown in workflow logs. Masked if accidentally printed.
4	Streamlit Cloud dashboard	Developer enters GLM_API_KEY in Settings → Secrets (TOML format)	HTTPS-only dashboard. Access requires GitHub OAuth login. Secret encrypted at rest by Snowflake.
5	Streamlit Cloud runtime	_secret('GLM_API_KEY') reads from st.secrets and assigns to GLM_API_KEY variable in memory	Key exists only in process memory. Never written to disk or logs.
6	Outbound HTTP request	GLM_API_KEY used as Authorization header in requests.post() call to Z.AI endpoint	Sent over HTTPS (TLS 1.2+). Header value never logged by application code.
7	Key rotation (if needed)	Update the value in Streamlit Cloud Secrets dashboard. App restarts automatically.	Old key invalidated at Z.AI dashboard. New key active after Streamlit restart (~30s).

3.4 Staff Credential Security

3.4.1 Current Implementation

Staff passwords are hashed using Python's built-in hashlib.sha256() before being stored in data/users.json. The plaintext password is never written to disk or stored in session state. Login validation compares the SHA-256 hash of the submitted password against the stored hash.

This implementation is appropriate for a hackathon prototype but has a known weakness: SHA-256 without a per-user cryptographic salt is vulnerable to rainbow table attacks if the users.json file is compromised. Two users with identical passwords will have identical hash values, making the weakness identifiable.

3.4.2 Known Weakness & Accepted Risk

Weakness	Risk Level	Why Accepted for Prototype	Production Fix
SHA-256 without salt	Medium	The user base is a small number of internal LRT staff. The users.json file is not exposed externally. The risk of targeted attack during a hackathon demo is negligible.	Replace hashlib.sha256 with bcrypt (work factor 12) or Argon2id. Add a per-user random salt (os.urandom(32)) stored alongside the hash in users.json.
Passwords stored in a JSON flat file	Low	Single-user write environment. No concurrent write risk during demo.	Migrate to PostgreSQL users table with proper schema: (staff_id, password_hash, salt, role, created_at, last_login).

Weakness	Risk Level	Why Accepted for Prototype	Production Fix
No session expiry or account lockout	Low	Duty manager sessions are managed within a single Streamlit session lifetime. The app is not exposed to brute-force login attempts in the demo.	Implement session JWT with 8-hour expiry. Lock account after 5 failed login attempts for 30 minutes.
No HTTPS enforcement at application level	Low	Streamlit Cloud provides HTTPS automatically via the .streamlit.app domain. The application itself does not need to configure TLS.	No change needed for Streamlit Cloud. For self-hosted production, HTTPS termination via NGINX reverse proxy.

3.5 Repository Security Controls

The following controls are applied at the repository level to prevent credential exposure:

Control	Implementation	What It Prevents
.gitignore — credential files	.env, .env.example, .streamlit/secrets.toml, *.db, data/users.json are all listed in .gitignore	Prevents API keys, secrets TOML, database files, and user credential files from being accidentally committed to the repository.
CI .env detection check	GitHub Actions ci.yml includes a find command that scans the entire repository tree for .env files and fails the build if any are found	Catches cases where a developer accidentally removes .env from .gitignore or creates a differently named secrets file.
GitHub Actions secret masking	Secrets injected via ${{ secrets.GLM_API_KEY }} are automatically masked in GitHub Actions log output (replaced with ***)	Prevents API keys from appearing in workflow run logs visible to repository collaborators.
No secrets in commit messages	Team convention — commit messages must never reference API key values	Prevents key values from appearing in git log, which is public for public repositories.
Branch protection (Section 2.4)	Direct pushes to main are blocked. All changes require a PR with CI passing.	Reduces the risk of a hasty direct push accidentally committing a secrets file.

3.6 Outbound API Integration Security

Both external API integrations (Z.AI GLM and HuggingFace DETR) use HTTPS exclusively. The following table documents the security properties of each integration:

Integration	Endpoint	Auth Method	Transport	Data Sent	Data Sensitivity
Z.AI GLM-5.1 / GLM-5-Turbo	https://api.z.ai/api/anthropic/v1/messages	Bearer token (GLM_API_KEY) in	HTTPS (TLS)	Schedule context, situation descriptio	Operational — no PII

Integration	Endpoint	Auth Method	Transport	Data Sent	Data Sensitivity
		Authorization header		n, historical precedent text	
HuggingFace DETR	https://api-inference.huggingface.co/models/facebook/detr-resnet-50	Bearer token (HF_API_KEY) in Authorization header	HTTPS (TLS)	CCTV image (JPEG/PNG, <5MB)	Visual — platform crowd image, no identifiable persons targeted

Note: No personally identifiable information (PII) is sent to any external API. GLM receives operational scheduling text only. HuggingFace receives crowd density images in which individual face recognition is not performed — only aggregate person counts are used.

4. Deployment Procedure

Follow the steps below in the order presented. Steps 1–4 are one-time setup. Step 5 is the ongoing update cycle.

Step 1 — Apply the st.secrets Code Change

Update core/glm_client.py to use the _secret() helper as documented in Section 3.2.3. Commit and push to a feature branch:

```
git checkout -b feat/streamlit-cloud-secrets
# Edit core/glm_client.py — add _secret() helper, replace os.getenv() calls
git add core/glm_client.py
git commit -m "feat: use st.secrets with os.getenv fallback for Streamlit Cloud"
git push origin feat/streamlit-cloud-secrets
```

Open a pull request against main. GitHub Actions CI will run automatically. Merge after CI passes and code review is complete.

Step 2 — Ensure data/ Directory is Committed

```
touch data/.gitkeep
git add data/.gitkeep
git commit -m "chore: add .gitkeep so data/ directory exists on cloud
deployment"
git push origin main
```

Step 3 — Connect Repository to Streamlit Cloud

6. Go to https://share.streamlit.io and sign in with your GitHub account.
7. Click New app.
8. Select repository: kahhhhjun/lrt-ai-operations.
9. Set Branch: main.
10. Set Main file path: lrt_ai/auth.py.
11. Click Deploy. Streamlit Cloud installs requirements and starts the app automatically.

Step 4 — Configure Secrets

In the Streamlit Cloud dashboard, navigate to the app → Settings → Secrets. Enter in TOML format:

```
GLM_API_KEY     = "your_z_ai_api_key_here"
GLM_ENDPOINT    = "https://api.z.ai/api/anthropic/v1/messages"
GLM_MODEL       = "glm-5.1"
GLM_MODEL_FAST  = "glm-5-turbo"
HF_API_KEY      = "your_huggingface_api_key_here"
```

Click Save. The app restarts and picks up the secrets automatically.

Step 5 — Ongoing Update Cycle

All subsequent code changes follow this automated cycle:

```
# 1. Create a feature branch
git checkout -b feat/your-feature
```

```
# 2. Make changes, commit
git add .
git commit -m "feat: your change description"
git push origin feat/your-feature

# 3. Open PR → GitHub Actions CI runs automatically
# 4. PR approved + CI passes → merge to main
# 5. Streamlit Cloud detects push to main → redeploys automatically
# No manual deploy step required.
```

5. Known Constraints on Streamlit Cloud

Constraint	Description	Impact on Demo
Ephemeral filesystem	SQLite DB and users.json reset on redeployment.	Register a test account after each fresh deployment. Expected behaviour.
App sleep	Free tier sleeps after ~7 days inactivity. ~20–30s wake time.	Access the app the day before the demo to ensure it is awake.
Single process	No background workers or concurrent write locking beyond SQLite.	No impact for demo — single evaluator session.
1 GB RAM limit	Shared vCPU, 1 GB RAM on free tier.	No impact — RAILMIND uses ~80–120 MB per session.
Public URL	App is publicly accessible at the .streamlit.app URL.	Protected by authentication. Share URL directly with judges.

6. Post-Deployment Verification

Check	How to Verify	Expected Result
App loads	Navigate to the Streamlit Cloud URL.	Login page renders. No error banner.
Registration works	Click Sign Up. Register a test account.	Account created. Login redirects to dashboard.
Schedule loads	Select a date and LRT line.	18-hour schedule table populates correctly.
GLM API active	Click Analyse with any input.	Phase 1 options appear instantly. Phase 2 GLM streams in ~8–14s.
Secrets configured	If GLM fails, check Settings → Secrets in Streamlit dashboard.	GLM_API_KEY is present and correct.
HTTPS active	Check browser address bar.	URL is https://. Padlock visible.
CI pipeline active	Open a test PR on GitHub. Check Actions tab.	RAILMIND CI workflow runs and passes.

7. Rollback Procedures

Scenario	Cause	Recovery Action
App fails to start after deploy	Syntax/import error in recent commit.	Check Streamlit Cloud Logs. git revert the breaking commit and push to main. CI will run, Streamlit will redeploy.
GLM calls fail	GLM_API_KEY missing or expired in Secrets.	Settings → Secrets → verify/update GLM_API_KEY. Save triggers automatic restart.
CI pipeline blocked	.env file accidentally committed.	Remove .env from the commit: git rm --cached .env && git commit -m 'remove .env'. Push fix.
Need to revert to previous version	Breaking change reached main.	git revert <commit-hash> && git push origin main. CI runs on the revert commit, Streamlit deploys the reverted state.
HuggingFace DETR unavailable	API down or HF_API_KEY expired.	CCTV feature degrades gracefully — all other features remain functional. Update HF_API_KEY in Secrets if expired.

8. Sign-Off

This Deployment Plan documents the complete deployment architecture for RAILMIND, addressing the judging criteria of Technical Stack & Scalability, CI/CD Pipeline & Automation, and Integration Security & Credential Management. All procedures have been verified against the live repository and Streamlit Cloud deployment prior to submission.

Role	Name / Team	Date
Prepared By	Wi-Five (GitHub: kahhhhjun)	3 May 2026
Submitted To	UMHackathon 2026 Judging Panel	Final Round
Version	**3.0 — Final (Streamlit Cloud)**	

End of Document